

NOTES

String Objects



String Objects

In Python, string objects are instances of the `str` class and represent sequences of characters. Python treats strings as immutable objects, meaning once a string is created, it cannot be changed. Any operation on a string that modifies its value will return a new string, leaving the original unchanged.

1. Creating Strings

Strings in Python can be created by enclosing characters in single (`'`) or double (`"`) quotes:

```
Python
my_string = "Aimerz!"
my_string2 = 'DATA SCIENCE WITH GEN AI'
```

Multi Line Strings

You can also use triple quotes (`'''` or `"""`) for multi-line strings:

```
Python
multi_line_string = """This is
a multi-line
string."""
```

2. String Methods

Python provides a variety of built-in methods for string manipulation. Here are some commonly used string methods:

Changing Case:

- `upper()`: Converts all characters to uppercase.
- `lower()`: Converts all characters to lowercase.
- `capitalize()`: Capitalizes the first letter of the string.
- `title()`: Capitalizes the first letter of each word.

```
Python
text = "hello world"
print(text.upper())    # "HELLO WORLD"
print(text.capitalize()) # "Hello world"
print(text.title())    # "Hello World"
```

Checking Properties:

- `isalpha()`: Returns True if all characters are alphabetic.
- `isdigit()`: Returns True if all characters are digits.
- `isalnum()`: Returns True if all characters are alphanumeric (letters or digits).

```
Python
text = "Python3"
print(text.isalnum()) # True
print(text.isdigit()) # False
print(text.isalpha()) # False
```

Searching and Replacing:

- `find(substring)`: Returns the index of the first occurrence of the substring, or -1 if not found.
- `replace(old, new)`: Replaces all occurrences of old with new.

```
Python
text = "hello world"
print(text.find("world")) # 6
print(text.replace("world", "Python")) # "hello Python"
```

Splitting and Joining:

- `split(separator)`: Splits the string into a list of substrings based on the separator.

- `join(iterable)`: Joins the elements of an iterable (like a list) into a string, using the string as the separator.

Python

```
text = "apple,banana,orange"
fruits = text.split(",") # ['apple', 'banana', 'orange']
print(", ".join(fruits)) # 'apple, banana, orange'
```

Trimming Whitespace:

- `strip()`: Removes leading and trailing whitespace.
- `lstrip()`: Removes leading whitespace only.
- `rstrip()`: Removes trailing whitespace only.

Python

```
text = "    hello world    "
print(text.strip()) # "hello world"
```

String Slicing:

Python allows slicing to extract substrings by specifying the start and end indexes.

Python

```
text = "Hello, World!"
print(text[0:5]) # "Hello"
print(text[7:]) # "World!"
print(text[-6:]) # "World!"
```

String Immutability

As mentioned, strings in Python are immutable. This means that any operation on a string that appears to modify it will actually return a new string. The original string remains unchanged:

```
Python
text = "Hello"
text.upper() # Returns "HELLO", but 'text' remains "Hello"
print(text) # "Hello"
```

String Formatting

Python provides various ways to format strings:

- **Using % operator (old style):**

```
Python
name = "Alice"
age = 25
print("My name is %s and I am %d years old." % (name, age))
```

Output:
My name is Alice and I am 25 years old.

- **Using str.format() method :**

```
Python
print("My name is {} and I am {} years old.".format(name, age))
```

Output:
My name is Alice and I am 25 years old.

- Using f-string for versions Python 3.6+:

```
Python
print(f"My name is {name} and I am {age} years old.")
```

Output:

```
My name is Alice and I am 25 years old.
```

Escape Sequences:

Python supports escape sequences within strings, which allow the inclusion of special characters like newlines (`\n`) or tabs (`\t`):

```
Python
text = "This is a newline character: \nAnd this is a tab character: \tHello!"
print(text)
```

Output:

```
This is a newline character:
And this is a tab character:  Hello!
```

3.String Operations

When performing operations between two Python strings, there are several types of operations you can use. Below are some examples of the most common operations that can be performed between two strings.

String Concatenation

- Combining two strings using the ``+`` operator.

```
Python
str1 = "Hello"
```

```
str2 = "World"

result = str1 + " " + str2 # Adding a space between them
print(result) # Output: 'Hello World'
```

- Using .join() method

```
Python
var1 = "Aimerz Pathway"
var2 = "Python"

# join() method is used to combine the strings
print("".join([var1, var2])) # Output: Aimerz PathwayPython

# the string with a separator Space(" ")
var3 = " ".join([var1, var2])
print(var3) # Output: Aimerz Pathway Python
```

String Repetition

Using the `` operator to repeat a string a certain number of times.

```
Python
str1 = "Ha"

result = str1 * 3
print(result) # Output: 'HaHaHa'
```

String Comparison

You can compare two strings using comparison operators (`==`, `!=`, `<`, `>`, `<=`, `>=`).

- `==` checks if two strings are equal.
- `<` checks if one string is lexicographically smaller than another.

```
Python
str1 = "apple"
str2 = "banana"

print(str1 == str2) # Output: False
print(str1 < str2)  # Output: True (because 'apple' comes before
                    'banana' lexicographically)
```

Checking Substring

Using the `in` operator to check if one string is a substring of another.

```
Python
string = "Hello, world!"
substring = "World"

if substring in string:
    print(f'"{substring}" is found in "{string}".')
else:
    print(f'"{substring}" is not found in "{string}".')

Output:
"World" is not found in "Hello, world!".
```

String Interpolation (F-Strings)

You can combine two strings with other values using formatted string literals (f-strings).

```
Python
str1 = "Hello"
str2 = "World"

result = f"{str1}, {str2}!"
print(result) # Output: 'Hello, World!'
```

Checking if Two Strings Are Anagrams

You can check if two strings are anagrams (contain the same characters) by sorting and comparing them.


```
Python
str1 = "listen"
str2 = "silent"

are_anagrams = sorted(str1) == sorted(str2)
print(are_anagrams) # Output: True
```

The `sorted()` function converts the string into a list of characters and sorts them in lexicographical (alphabetical) order.

After sorting:

- `sorted(str1) → ['e', 'i', 'l', 'n', 's', 't']`
- `sorted(str2) → ['e', 'i', 'l', 'n', 's', 't']`

Replacing Characters Between Two Strings

Replacing all occurrences of characters from one string with another string's characters.

```
Python
str1 = "hello"
str2 = "world"

# Replace 'l' in str1 with 'r' from str2
result = str1.replace('l', str2[2])
print(result) # Output: 'herro'
```

Steps:

1. `str1` is "hello", and `str2` is "world".
2. You want to replace all occurrences of the letter 'l' in `str1` with the third character (index 2) of `str2`.
3. In Python, indexing starts at 0, so `str2[2]` gives you the character 'r' because:
 - `str2[0]` is 'w'
 - `str2[1]` is 'o'
 - `str2[2]` is 'r'

4. The `replace()` function is used to replace all occurrences of a substring in `str1`. Here, the substring 'l' is replaced by the character 'r'.
5. The `replace()` function works as follows:
 - It looks for all occurrences of the letter 'l' in `str1` ("hello").
 - Then, it replaces each 'l' with 'r'.
6. After the `replace()` operation, all occurrences of 'l' in "hello" are replaced by 'r'. The string changes from "hello" to "herro".
7. The output is "herro".

Conclusion

Understanding the concepts of string operations like concatenation, repetition, comparison, substring checks, common prefixes, merging, anagrams, and character replacement.

THANK YOU

